Technical Document

Manage Ticket Tracking — SRS

SOFTWARE REQUIREMENTS SPECIFICATION

Manage Ticket Tracking · Sprint Board

Software Requirements Specification (SRS) — Manage Ticket Tracking

Feature: Manage Ticket Tracking (Sprint board) — Apex controller: force-app/main/default/classes/controller/manageTicketTracking/ManageTicketTrackingController.cls

1. Purpose & Scope

The Manage Ticket Tracking page is the sprint-execution surface of the Jira Clone. For the single active sprint of a selected Project it lets a user:

- See the active sprint header (name, date range, goal) and a live story-point progress line.
- See one board column per project status, each holding the tickets currently in that status.
- Read every ticket as a card showing its name, ticket type, summary, story points, and assignee.
- Move a ticket from one status column to another by drag-and-drop, limited to the transitions allowed for that ticket's type.
- Open a ticket in a centered ticket-view pop-up to edit its summary, status, and description.
- Manage a ticket's linked work items, sub-tasks, history, and comments from the ticket-view pop-up.

The project is selected once and persisted in localStorage('projectId'); if absent the page shows the Choose Project splash. Only the sprint whose status is in_progress is loaded; if there is none, a "No active sprint" banner is shown.

2. Actors

Actor	Description
Project Member (User)	The authenticated Salesforce user working the active sprint — moves tickets across status columns and edits ticket details (summary, status, description, links, sub-tasks, comments).

3. Functional Requirements

3.1 Project Selection and Data Loading

- FR-1 The system shall allow a user to select a project before accessing the ticket-tracking board.
- FR-2 The system shall load the board data for the selected project: statuses, workflow transitions, members, ticket types, the active sprint, and that sprint's active tickets.

- FR-3 The system shall load only the sprint whose lifecycle state is in_progress; if none exists, it shall show a “No active sprint” banner instead of the board.

3.2 Sprint Header and Progress

- FR-4 The system shall display the active sprint’s name, start → end date range (end derived from start date + duration), and goal.
- FR-5 The system shall display a story-point progress line showing ended story points / total story points and the completion percentage.

3.3 Board Columns and Ticket Cards

- FR-6 The system shall render one column per project status; the column header shall show the status name and the count of tickets it holds.
- FR-7 Each column shall hold 0..many ticket cards — the tickets whose current status equals that column’s status — ordered by score.
- FR-8 Each ticket card shall show the ticket name, ticket type, summary, story points, and assigned project-member name.
- FR-9 A ticket that has reached an end (Done) status shall be marked with a green left border.

3.4 Ticket Movement (Status Change)

- FR-10 The system shall allow users to move a ticket to another status column by drag-and-drop.
- FR-11 On drag start the system shall highlight only the columns that are valid targets for that ticket, computed from the ticket type’s workflow transitions out of its current status (so a Task and a Story expose different reachable columns).
- FR-12 Dropping on a non-target column or on the ticket’s own column shall be ignored.
- FR-13 On drop the system shall apply the status change only if a workflow transition exists for fromStatus → toStatus and all validation-field rules attached to that transition pass; otherwise the move is rejected with a message and the card stays in place.
- FR-14 When a move takes a ticket to an end status, the sprint’s ended story points and the progress line shall update.

3.5 Ticket View (Pop-up)

- FR-15 The system shall allow users to open a ticket in a centered modal ticket-view pop-up by clicking its card.
- FR-16 The system shall allow users to close the ticket view and return to the board.
- FR-17 The system shall allow users to edit the ticket summary from the ticket view.
- FR-18 The system shall allow users to change ticket status from the ticket view, subject to the same workflow + validation-field rules as a board move.
- FR-19 The system shall allow users to edit the ticket description using a rich-text editor.

3.6 Linked Work Items

- FR-20 The system shall allow users to expand a ticket’s linked work items in the ticket view.

• FR-21 The system shall allow users to add a link by choosing a link type and searching for a target ticket (minimum 2 characters).

3.7 Sub-tasks

• FR-22 The system shall allow users to expand a ticket's sub-tasks in the ticket view.
• FR-23 The system shall allow users to create a sub-task with summary (required) plus optional assignee, state, start date, story points, and description.

3.8 History and Comments

• FR-24 The system shall allow users to expand a ticket's history (activity feed) in the ticket view.
• FR-25 The system shall allow users to expand and read a ticket's comments.
• FR-26 The system shall allow users to add a comment to a ticket.

4. Validation Rules

4.1 Client-side (LWC validators)

• VR-1 A status change requires a ticketId and a toStatusId (validateChangeTicketState) before any Apex call.
• VR-2 A drop is accepted only on a column flagged as a valid target; a drop on the ticket's current column is a no-op.
• VR-3 A board move is allowed only when a matching workflow transition is found on the ticket type (findTransitionId); otherwise "This transition is not allowed by the workflow." is shown.
• VR-4 Creating a sub-task requires a non-blank Summary.
• VR-5 Linking requires both a link type and a target ticket; ticket search ignores terms shorter than 2 characters.

4.2 Server-side (controller input guards)

• VR-6 Every @AuraEnabled method blank-checks its required parameters and returns APIResponse(false, '<field> is required') before doing any work (e.g. projectId, ticketId, fromStatusId, toStatusId, summary, message, fromTicketId, toTicketId, linkType).
• VR-7 Referenced records must exist via DomainCorrectness.require* (project, ticket, ticket type, workflow, from/to status); optional references (assignee, sub-task state) use requireOptional*.
• VR-8 loadTicketBySearchTerm escapes all SOSL/SOQL special characters in the search term before building the query.

5. Business Rules

5.1 Active sprint selection

• BR-1 The board loads exactly one sprint — the project's sprint whose RecordStatus__c = 'in_progress' (at most one per project); if none, the board shows no sprint.

• BR-2 Only active tickets (RecordStatus__c = 'active') of that sprint are loaded, ordered by Score__c ASC.

5.2 Columns & cards

• BR-3 Columns are derived from the project's statuses (in their defined order); a ticket appears in the column whose status equals the ticket's CurrentState__c.
• BR-4 A column can hold 0..many cards; the header count reflects the number of cards in that column.
• BR-5 A ticket whose current status has isEnd__c = true is an "ended" ticket and is rendered with a green left border.

5.3 Transitions & validation

• BR-6 A status change is allowed only if a WorkflowTransition__c exists for fromStatus → toStatus on the workflow of the ticket's type (requireTransitionAllowed).
• BR-7 Every ValidateField__c rule attached to that transition must pass (requireValidateFieldsPass) before the status is written; a failing rule rejects the change with the rule's error message.
• BR-8 The set of reachable columns is therefore ticket-type specific — two types pointing at different workflows expose different valid targets from the same status.
• BR-9 An end status is terminal: no transition may originate from it.

5.4 Story-point accounting

• BR-10 The sprint's story-point line shows TotalEndedStoryPoint__c / TotalStoryPoint__c and a percentage = round(ended / total × 100).
• BR-11 When a ticket in the sprint reaches an end status, its story points are added to the sprint's ended total and the progress line refreshes.

5.5 Data hygiene

• BR-12 All ticket, sub-task, link, history, and comment reads exclude soft-deleted rows (RecordStatus__c = 'active' / != 'deleted'), per the project's soql-exclude-deleted rule.

6. Use Case Descriptions

UC-1 — Load the board

• Actor: Project Member
• Pre-conditions: User is authenticated; a projectId exists in localStorage.
• Main flow:
1. Component mounts and reads projectId. 2. Calls loadManageTicketTrackingPage(projectId). 3. Renders the sprint header, the story-point line, and one column per status filled with the active sprint's tickets.
• Alternative flows:
• A1 (no project): No projectId → Choose-Project splash; on projectchosen the project is stored and the main flow resumes from step 2.

- A2 (no active sprint): The sprint is null → “No active sprint. Start a sprint from the Backlog page.” banner; columns are not shown.
- A3 (load error): Apex returns success=false or throws → error toast, content hidden.

UC-2 — Move a ticket between columns

- Actor: Project Member
- Pre-conditions: The board is loaded with an active sprint and at least two statuses.
- Main flow:

1. User starts dragging a ticket card. 2. The system highlights the columns reachable from the ticket’s current status for that ticket type. 3. User drops on a highlighted column. 4. changeTicketState(ticketId, fromStatusId, toStatusId) validates the transition and its validation-field rules, writes the new status, and — if the new status is an end status — updates the sprint’s ended story points. 5. The card moves to the target column.

- Alternative flows:
- A1 (non-target column): Drop is ignored; the card stays.
- A2 (same column): No-op.
- A3 (transition not allowed / validation-field fails): Error toast; the card stays in place.

UC-3 — Open the ticket view

- Actor: Project Member
- Pre-conditions: A ticket card is visible on the board.
- Main flow:

1. User clicks a ticket card. 2. ticketviewopen records the active ticket Id. 3. A centered modal renders the ticket: its TKT-id, summary, current status, description, and the Linked work items / Sub-tasks / History / Comments sections.

- Alternative flows:
- A1 (close): Clicking the backdrop or the × clears the active ticket and unmounts the panel.

UC-4 — Edit summary, status, or description from the ticket view

- Actor: Project Member
- Pre-conditions: The ticket view is open.
- Main flow (any one of):
- Edit summary → ticketsummaryupdate → updateTicketSummary.
- Change status (combo) → ticketstatuschange → changeTicketState (BR-6 / BR-7).
- Edit description → click the description → rich-text editor → Save → ticketdescriptionupdate → updateTicketDescription.
- Alternative flows:
- A1 (server error on any action): Error toast; the panel keeps its last good value.

UC-5 — Link work items

- Actor: Project Member
- Pre-conditions: The ticket view is open.
- Main flow:

1. User expands Linked work items (ticketlinkedtoexpand → loadTicketLinkedTo lists existing links). 2. User clicks +, chooses a link type, and searches a target ticket (ticketsearch → loadTicketBySearchTerm). 3. User clicks Link → linkToTicket adds the link to the list.

- Alternative flows:
- A1 (incomplete): Link stays disabled until both a type and a target ticket are chosen; searches under 2 characters are ignored.

UC-6 — Create and expand sub-tasks

- Actor: Project Member
- Pre-conditions: The ticket view is open.
- Main flow:

1. User expands Sub-tasks (subtasksexpand → loadSubtasks). 2. User clicks +, fills the form (Summary required; optional assignee, state, start date, story points, description). 3. Create → createSubtask adds the sub-task to the list.

- Alternative flows:
- A1 (summary blank): Required-field validation; no call.
- A2 (cancel): The form closes.

UC-7 — View ticket history

- Actor: Project Member
- Pre-conditions: The ticket view is open.
- Main flow:

1. User expands History (tickethistoryexpand → loadTicketHistory). 2. The system lists the ticket's activity feed, each entry with its author and timestamp.

UC-8 — Read and add comments

- Actor: Project Member
- Pre-conditions: The ticket view is open.
- Main flow:

1. User expands Comments (ticketcommentsexpand → loadTicketComments). 2. User types a comment and clicks Comment → createTicketComment prepends it to the list.

- Alternative flows:
- A1 (re-expand): Re-expanding the same ticket refreshes the comments via refreshApex.

7. Non-Functional Notes

- Read efficiency: the page aggregate, linked-items, search, sub-tasks, history, and comments reads are cacheable=true; linked items, sub-tasks, history, and comments load lazily on expand.
- Derived UI state: the LWC keeps one principal state (the sprint with its tickets) and derives every displayed value — columns, valid drop targets, the active ticket-view model, the story-point percentage — via getters and formatter utilities, never parallel tracked fields.

- Type-aware transitions client-side: valid drop targets are computed in the browser from each ticket type's nested workflow transitions, so no round-trip is needed to highlight reachable columns.
- Soft delete everywhere: tickets, sub-tasks, links, and comments are soft-deleted; queries filter them out.